difference between `var` hoisting and the Temporal Dead Zone (TDZ) behavior for `let` and `const`. It also assesses understanding of how JavaScript resolves variables during the creation and execution phases of a function.

**Answer:** This code throws a ReferenceError: `ReferenceError: Cannot access 'message' before initialization`. Although `let` is hoisted, unlike `var`, it is not initialized during hoisting. The variable exists in the Temporal Dead Zone (TDZ)—the period between entering its scope and the `let` declaration—where any attempt to access it results in a `ReferenceError`.

By contrast, `var` is hoisted and initialized to `undefined` at the top of its scope. This allows access before its declaration, though the value will be `undefined` until assignment.

#### TIP

Removing 'use strict' does not change this behavior. The TDZ for let and const is enforced by the ES6 specification, regardless of strict mode.

For this topic, are you able to answer these follow-up/probing questions?

- "How would refactoring legacy code that uses `var` to use `let` and `const` improve code quality? What pitfalls should you watch for?"
- "Explain the Temporal Dead Zone (TDZ). Why was it introduced, and how does it affect error handling?"
- "How does hoisting behave differently for variables declared with `var` versus `let`/`const`?"
- "In what scenarios might variable leakage occur in JavaScript, and how can it be avoided?"
- "How do variables declared in different scopes (function, block, global) interact, and what common pitfalls arise?"

**Question (applied skill):** "Can you trace the scope chain and explain why count persists across calls, as demonstrated in listing 3.2?"

#### Listing 3.2 Closure—tracing the scope chain and persistence

```
function outer() {
```